微算機實習**Lab09**

組別:第**14**組

資工三 **112590034** 高翎竣
資工三 **112590032** 陳俊彥
資工三 **112590453** 詹東儒

日期:**114**年**12**月**19**日

一、實驗內容

這次實驗要透過學習啟動兩種不同的執行緒，監控(任務)以及大量運算(資料)來同時執行不同性質的任務。透過mutex來保護計數器，並透過semaphore來控制LED顯示。

二、實驗過程及結果

I. 實驗過程：

這次實驗要透過學習啟動兩種不同的執行緒，監控(任務)以及大量運算(資料)來同時執行不同性質的任務。透過mutex來保護計數器，並透過semaphore來控制LED顯示。

II. 實驗結果：

透過**啟**動執行緒並對其進行操作、監控，來進行1～50000的平行運算，在開始運算的時候會先減慢速度讓LED有明顯的變化，之後再以正常速度來運行並輸出結果。

III. 遇到的問題以及怎麼解決：

我們當初在寫Lab09的時候不知道該如何開頭，總感覺怎麼寫都怪怪的，結果有組員提議延續用Lab08的驅動程式來做Lab09，沒想到意外地順利。

三、程式碼

Makefile:

```
obj-m := demo.o

kernel_dir := /usr/src/linux-headers-4.9.201-tegra-ubuntu18.04_aarch64/kernel-4.9

PWD := $(shell pwd)

all:
	make -C $(kernel_dir) SUBDIRS=$(PWD)
clean:
	rm -f *.o *.ko *.mod.c
.PHONY:
	clean
```

demo.c:

```c
#include <linux/init.h>
#include <linux/kernel.h>
#include <linux/module.h>
#include <linux/fs.h>
#include <linux/string.h>
#include <linux/uaccess.h>
#include <asm/uaccess.h>

#define MAJOR_NUM 60
#define MODULE_NAME "demo"

static int iCount = 0;
static char userChar[100];

//  LED status
static char led_status[4] = {'0', '0', '0', '0'}; //  LED1 ~ LED4

// Open function
static int drv_open(struct inode *inode, struct file *filp) {
        printk("Enter Open function\n"); // [cite: 303]
        return 0;
}

// Release function
static int drv_release(struct inode *inode, struct file *filp) {
        printk("Enter Release function\n"); // [cite: 304]
        return 0;
}


static void file_write(char *path, char *str) {
    struct file *fp;
    loff_t pos = 0;
    mm_segment_t old_fs;

    old_fs = get_fs();
    set_fs(get_ds());

    fp = filp_open(path, O_WRONLY, 0);
    if (IS_ERR(fp)) {
        printk("File open error: %s\n", path);
    } else {
        vfs_write(fp, str, strlen(str), &pos);
        filp_close(fp, NULL);
    }

    set_fs(old_fs);
}

// Read function : return LED status to User
static ssize_t drv_read(struct file *filp, char *buf, size_t count, loff_t *ppos) {
    printk("Enter Read function\n");

    // [Fix] let led_status copy to user's buf
    // return recorded 4 LED status for real time
    int ret;
    if (count < 4) return -EINVAL; // Buffer 2 small

    ret = copy_to_user(buf, led_status, 4);
    if (ret != 0) {
        return -EFAULT;
    }

    return 4; // return loaded 4 bytes
}
```

```c
// Write function：receive User operation then control LED
static ssize_t drv_write(struct file *filp, const char *buf, size_t count, loff_t *ppos) {
        printk("device write\n");
        printk("%d\n", iCount);
        printk("W_buf_size: %d\n", (int)count);
        if(count > 99){ count = 99; }   //

        copy_from_user(userChar, buf, count);
        userChar[count] = '\0';
        printk("userChar(char): %s\n", userChar);

        char *gpio_path_led1 = "/sys/class/gpio/gpio396/value";         //pin 7
        char *gpio_path_led2 = "/sys/class/gpio/gpio466/value";         //pin 11S
        char *gpio_path_led3 = "/sys/class/gpio/gpio297/value";         //pin 32
        char *gpio_path_led4 = "/sys/class/gpio/gpio254/value";         //pin 22

        if (strncmp(userChar, "LED1on", 6) == 0) {
                file_write(gpio_path_led1, "1");
                led_status[0] = '1';
                printk("LED1 turned ON\n");
        } else if (strncmp(userChar, "LED1off", 7) == 0) {
                file_write(gpio_path_led1, "0");
                led_status[0] = '0';
                printk("LED1 turned OFF\n");
        } else if (strncmp(userChar, "LED2on", 6) == 0) {
                file_write(gpio_path_led2, "1");
                led_status[1] = '1';
                printk("LED2 turned ON\n");
        } else if (strncmp(userChar, "LED2off", 7) == 0) {
                file_write(gpio_path_led2, "0");
                led_status[1] = '0';
                printk("LED2 turned OFF\n");
        } else if (strncmp(userChar, "LED3on", 6) == 0) {
                file_write(gpio_path_led3, "1");
                led_status[2] = '1';
                printk("LED3 turned ON\n");


        } else if (strncmp(userChar, "LED3off", 7) == 0) {
                file_write(gpio_path_led3, "0");
                led_status[2] = '0';
                printk("LED3 turned OFF\n");
        } else if (strncmp(userChar, "LED4on", 6) == 0) {
                file_write(gpio_path_led4, "1");
                led_status[3] = '1';
                printk("LED4 turned ON\n");
        } else if (strncmp(userChar, "LED4off", 7) == 0) {
                file_write(gpio_path_led4, "0");
                led_status[3] = '0';
                printk("LED4 turned OFF\n");
        }

        iCount++;
        return count;
}

// I/O Control function
static long drv_ioctl(struct file *filp, unsigned int cmd, unsigned long arg) {
        printk("Enter I/O Control function\n"); // [cite: 307]
        return 0;
}

// define file_operations [cite: 1013]
struct file_operations drv_fops = {
        .read = drv_read,
        .write = drv_write,
        .unlocked_ioctl = drv_ioctl,
        .open = drv_open,
        .release = drv_release,
};
```

```c
// Init function [cite: 1028]
static int demo_init(void) {
        if (register_chrdev(MAJOR_NUM, MODULE_NAME, &drv_fops) < 0) {
                printk("%s: can't get major %d\n", MODULE_NAME, MAJOR_NUM);
                return -EBUSY;
        }
        printk("<1>%s: started\n", MODULE_NAME);
        return 0;
}

// Exit function [cite: 1040]
static void demo_exit(void) {
    unregister_chrdev(MAJOR_NUM, MODULE_NAME);
    printk("<1>%s: removed\n", MODULE_NAME);
}

module_init(demo_init);
module_exit(demo_exit);
MODULE_LICENSE("GPL");
```

Lab09.cpp:

```cpp
#include <iostream>
#include <pthread.h>
#include <unistd.h>
#include <fcntl.h>
#include <string.h>
#include <stdint.h>

using namespace std;

#define DEVICE_PATH "/dev/demo"

volatile bool is_calculating = true;
int counter = 0;
pthread_mutex_t count_mutex = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t led_mutex = PTHREAD_MUTEX_INITIALIZER;
int fd;

void control_led(const char* cmd) {
    pthread_mutex_lock(&led_mutex);
    write(fd, cmd, strlen(cmd));
    pthread_mutex_unlock(&led_mutex);
}

// ==========================================
// task 1 Data Parallelism
// ==========================================

// Worker Thread 1:  1 ~ 25,000,000
void* thread_sum1(void* arg) {
    long long local_sum = 0;
    for (long long i = 1; i <= 25000000; ++i) {
        local_sum += i;

        // pause 1000 times * 10ms = 10000ms = 10秒
        if (i % 25000 == 0) {
            usleep(10000);
        }
    }

    pthread_exit((void*)local_sum);
}
```

```c
// Worker Thread 2:  25,000,001 ~ 50,000,000
void* thread_sum2(void* arg) {
    long long local_sum = 0;
    for (long long i = 25000001; i <= 50000000; ++i) {
        local_sum += i;

        if (i % 25000 == 0) {
            usleep(10000);
        }
    }

    pthread_exit((void*)local_sum);
}

// Monitor Thread (T3): Heartbeat - LED 1
void* thread_monitor(void* arg) {
    while (is_calculating) {
        control_led("LED1on");
        // 0.5 sec (50 * 10ms)
        for (int i = 0; i < 50 && is_calculating; ++i) {
            usleep(10000);  // 10ms
        }

        if (!is_calculating) break;

        control_led("LED1off");
        // 0.5 sec (50 * 10ms)
        for (int i = 0; i < 50 && is_calculating; ++i) {
            usleep(10000);  // 10ms
        }
    }
    control_led("LED1on");
    pthread_exit(NULL);
}


// ==========================================
// task 2
// ==========================================

void* thread_increment(void* arg) {
    // int thread_id = *(int*)arg;

    for (int i = 0; i < 10000; ++i) {
        pthread_mutex_lock(&count_mutex);

        // === Critical Section  ===
        // LED 2 turn on
        write(fd, "LED2on", 6);

        // delay
        usleep(100);

        counter++;

        // LED 2 turn off
        write(fd, "LED2off", 7);
        // === Critical Section  ===

        pthread_mutex_unlock(&count_mutex);
    }

    pthread_exit(NULL);
}
```

```
// ============================================
// main
// ============================================

int main() {
    fd = open(DEVICE_PATH, O_RDWR);
    if (fd < 0) {
        perror("can not open /dev/demo, use dmesg check ur driver");
        return -1;
    }

    // initialize LED status
    control_led("LED1off");
    control_led("LED2off");

    cout << "========================================" << endl;
    cout << "Starting Item 1: Parallel Summation & Heartbeat" << endl;
    cout << "(Will run for approx. 10 seconds to show LED blinking)" << endl;
    cout << "========================================" << endl;

    pthread_t t1, t2, t3;
    void* result1;
    void* result2;

    // Monitor Thread (T3)
    pthread_create(&t3, NULL, thread_monitor, NULL);

    // Worker Threads (T1, T2)
    pthread_create(&t1, NULL, thread_sum1, NULL);
    pthread_create(&t2, NULL, thread_sum2, NULL);

    pthread_join(t1, &result1);
    pthread_join(t2, &result2);

    // T3 turn off
    is_calculating = false;
    pthread_join(t3, NULL);


    long long total_sum = (long long)(intptr_t)result1 + (long long)(intptr_t)result2;

    cout << "T1 Result (1~25000000): " << (long long)(intptr_t)result1 << endl;
    cout << "T2 Result (25000001~50000000): " << (long long)(intptr_t)result2 << endl;
    cout << "Total Sum: " << total_sum << endl;
    cout << "Expected:  1250000025000000" << endl;
    cout << "LED 1 is now Always ON (Calculation Complete)" << endl;
    cout << endl;

    sleep(2);

    // ========== task 2 ==========
    cout << "========================================" << endl;
    cout << "Starting Item 2: Mutex Resource Protection" << endl;
    cout << "========================================" << endl;

    pthread_t t4, t5;
    int id1 = 1, id2 = 2;

    // reset counter
    counter = 0;

    cout << "Entering Critical Section..." << endl;

    pthread_create(&t4, NULL, thread_increment, &id1);
    pthread_create(&t5, NULL, thread_increment, &id2);

    pthread_join(t4, NULL);
    pthread_join(t5, NULL);

    cout << "Leaving Critical Section..." << endl;
    cout << "Final Counter: " << counter << " (Expected: 20000)" << endl;

    if (counter == 20000) {
        cout << "Mutex protection successful!" << endl;
    } else {
        cout << "WARNING: Counter mismatch! Race condition detected." << endl;
    }
```

```
    cout << endl;
    cout << "========================================" << endl;
    cout << "Experiment Complete!" << endl;
    cout << "========================================" << endl;

    // turn off all of LED
    control_led("LED1off");
    control_led("LED2off");

    pthread_mutex_destroy(&count_mutex);
    pthread_mutex_destroy(&led_mutex);

    close(fd);
    return 0;
}
```

四、本次實驗過程說明與解決方法

這次實驗中一開始花了很多時間研究如何啟動多個執行緒、分配不同的任務。使用mutex保護計數器，最後透過semaphore來操控輸出的LED。理解大略的定義以及操作方法之後，加上一點構思以及debug的時間，就順利的完成LAB了。

五、分工

學號、組員	貢獻比例	工作內容
112590034 高翎竣	33%	程式撰寫除錯、報告撰寫
112590032 陳俊彥	33%	程式撰寫除錯、報告撰寫
112590453 詹東儒	33%	程式撰寫除錯、報告撰寫